

End-to-End Orchestration Challenge

Coded App + RPA + Maestro — Bank Developer Edition

Build a single, working automation that spans the full UiPath platform: a **Coded App** front end, an **RPA workflow** that does the work, and a **Maestro (BPMN)** process that orchestrates it all end to end. This document sets the common baseline every submission must meet, the bonus opportunities that separate the leaders, and exactly how entries will be judged.

THEME	FORMAT	MAX BASE SCORE	DEADLINE
Support Ticket Dashboard	Individual	100 pts + up to 25 bonus	22 July 2026

1. The Challenge at a Glance

You will design and deliver a Support Ticket Dashboard as a UiPath Coded App, backed by data in UiPath Data Fabric, and wire it to a real automation that runs end to end. The point of the exercise is not just to build a nice UI — it is to prove you can orchestrate the full platform so that an action in the app flows through Maestro, triggers an RPA workflow, and writes results back to the data layer that the app then reflects.

Every submission must contain three connected building blocks:

- **Coded App** — the operational front end support leads use to view, search, and investigate tickets, and to kick off the process.
- **RPA workflow** — the automation that performs the actual work (enrichment, assignment, status updates, notifications).
- **Maestro / BPMN process** — the orchestration layer that models the ticket lifecycle and coordinates the app, the robot, and the data.

A submission that only builds a beautiful app but does not orchestrate end to end will not clear the baseline. A submission that orchestrates the full loop — even with a plain UI — will.

2. Business Scenario

Support teams rely on timely visibility into ticket activity to maintain service levels and resolve issues efficiently. Ticket data lives in UiPath Data Fabric, but support leads often lack a dedicated operational interface that lets them view all active tickets, search and sort across them, and investigate the detail of a single ticket.

Your job is to close that gap. Build a centralised operational view over ticket data that lets support leads investigate issues, create new tickets, and monitor team performance from one place — and, crucially, act on a ticket in a way that sets a real automated process in motion behind the scenes.

Suggested lifecycle to model (you may adapt it): Ticket raised → Auto-triage & categorise → Route / assign → Enrich & act → Resolve → Close, with SLA checks along the way.

3. Reference Architecture

These are the four layers your solution is expected to touch. How you connect them is part of

what is judged.

Coded App	The TypeScript/React operational dashboard. Reads and writes Data Fabric, and starts the orchestration.
Data Fabric	The system of record. Holds the Ticket entity (and any related entities) that both the app and the automation read and update.
Maestro (BPMN)	The orchestration brain. A BPMN process modelling the ticket lifecycle, coordinating human steps, automated steps, and decisions.
RPA workflow	The hands. A published automation invoked by Maestro (or the app) that performs concrete work and updates the data.

4. Baseline Requirements (Mandatory)

This is the common floor. A submission that meets every item below is eligible to win. Anything missing here disqualifies the entry regardless of polish elsewhere — so lock the baseline first, then reach for bonus points.

4.1 Coded App

☐	Built with UiPath Coded Apps (TypeScript). Runs and connects to your UiPath environment.
☐	Displays all active tickets in a list or grid, reading live from Data Fabric.
☐	Supports search and sort across tickets (e.g. by status, priority, assignee, or date).
☐	A ticket detail view that shows the full record when a ticket is selected.
☐	Ability to create a new ticket that persists to Data Fabric.
☐	An in-app action (e.g. a button on a ticket) that starts the orchestration.

4.2 Data Fabric

☐	A Ticket entity with, at minimum: ID, subject, description, priority, status, assignee, created date.
☐	At least one status field that the automation updates during the process (proving the write-back loop).

4.3 RPA Workflow

☐	At least one published RPA workflow that performs a meaningful task (e.g. enrichment, auto-assignment, status update, or notification).
☐	The workflow reads from and/or writes to the Data Fabric Ticket entity.
☐	The workflow is invoked as part of the orchestration — not run manually in isolation.

4.4 Maestro / BPMN Orchestration

☐	A Maestro process defined in BPMN with a start event, at least three tasks, at least one decision gateway, and an end event.
☐	The process models a recognisable ticket lifecycle (triage / assign / resolve / close or similar).
☐	The process invokes the RPA workflow as one of its steps.
☐	The process reads and/or writes Data Fabric so state is shared across the solution.

4.5 End-to-End Integration

☐	The full loop works: an action in the Coded App starts the Maestro process, which drives the RPA workflow, which updates Data Fabric, and the app reflects the change.
☐	You can demonstrate the loop live (or on video) from trigger to visible result.

5. Bonus Points (Optional — up to +25)

Bonus points are added on top of your base score. Pick the ones that showcase your strengths; you do not need all of them. Bonus is capped at +25 in total.

Agentic / AI	An agent that auto-categorises, prioritises, summarises, or recommends a resolution for a ticket, invoked within the flow.	+6
Document IDP	Use Intelligent Document Processing to extract data from a ticket attachment (e.g. a form or screenshot) and feed it into the process.	+4
Human-in-the-loop	A Maestro user task / Action Center step for approval or exception handling, with the app or email surfacing it.	+4
SLA & escalation	SLA timers with automatic escalation or re-routing when a threshold is breached.	+3
Notifications	Real notifications on key events (Email, Teams, or Slack) driven by the process.	+2
Role-based views	Distinct experiences for e.g. agent vs supervisor vs customer within the Coded App.	+3
Analytics / KPIs	An in-app dashboard with real metrics (open by priority, avg resolution time, SLA compliance).	+3

Engineering quality	Meaningful automated tests, clean reusable components, and robust error handling / BPMN compensation.	+3
Polished UX	A genuinely well-crafted, on-brand interface that a support lead would happily use daily.	+2

Note: bonus items only count once the relevant baseline is solid. A slick AI agent bolted onto a broken end-to-end loop earns nothing until the loop works.

6. Judging Criteria

Every qualifying submission is scored out of 100 against the rubric below, then bonus points are added. Judges score each criterion on how fully and how well it is delivered, not merely whether it is present.

Functional completeness & end-to-end flow	The full loop works reliably from app action through Maestro and RPA to a visible data update. This is the single most important criterion.	25
Orchestration & architecture	Quality of the Maestro BPMN design: sensible lifecycle, correct gateways, clean coordination of human, automated, and data steps; how well the four layers fit together.	20
Technical implementation & code quality	Sound Coded App and RPA build — readable, maintainable code, good use of the TypeScript SDK, sensible data model, no fragile hacks.	20
User experience & interface	Clarity and usability of the dashboard: can a support lead view, search, investigate, and act without friction.	10
Innovation & business value	Creativity and how convincingly the solution would help a real support team at a bank like Bank.	15
Documentation & demo	A clear README/architecture note and a crisp walkthrough that lets a judge understand and reproduce the flow.	10
Total	Base score before bonus points	100
Bonus	Section 5 opportunities, added on top	+25

Tie-breaker: where scores are level, the entry with the more robust, better-documented end-to-end orchestration wins.

7. What to Submit

Package everything below and submit by the deadline. Incomplete packages cannot be fairly judged.

☐	Link or export of the Coded App project (source included).
☐	The published RPA workflow (package or repo link).
☐	The Maestro / BPMN process definition.
☐	Data Fabric entity schema (export or screenshots of the entity design).
☐	A README covering: architecture overview, how the four layers connect, setup/run steps, and which bonus items you attempted.
☐	A demo video (5–8 minutes) walking through the end-to-end flow from trigger to result.
☐	A short architecture diagram showing App → Maestro → RPA → Data Fabric and back.

8. Timeline, Eligibility & Support

Timeline

- Kick-off & briefing — Done
- Design window — 21st July to 24th July 2026
- Submission deadline — 24 July 2026
- Judging & walkthroughs — 27th July 2026
- Winners announced — TBD

Eligibility & rules

- Open to Bank developers participating in the programme. Individual or teams of two.
- All work must be your own and built within your assigned UiPath environment / trial tenant.
- You may use UiPath documentation, community resources, and AI assistants; you must be able to explain your solution.
- Submissions after the deadline are not scored. One submission per person or team.

Prizes

1st Prize USD 50 Voucher

2nd Prize USD 30 Voucher

3rd Prize USD 20 Voucher

Support & resources

- UiPath Coded Apps and TypeScript SDK documentation.

- [Maestro / BPMN orchestration and Data Fabric entity guides.](#)